

# 15

## المهام التكرارية: مدخل إلى الحلقات

نستفيد من البرامج الحاسوبية خير استفادة في أتمتة المهام وإجراء المهام التكرارية لكيلا نحتاج إلى القيام بها يدويًا، وإحدى طرائق تكرار المهام المتشابهة هي استخدام حلقات التكرار (loops)، وسنشرح في هذا الفصل حلقتي التكرار الشهيرتين في بايثون -وسائر لغات البرمجة- `for` و `while`، وكيفية استعمالهما.

## 1. حلقة التكرار `while`

حلقة التكرار `while` تؤدي إلى تكرار تنفيذ قسم من الشيفرة بناءً على متغير منطقي (boolean)، وسيستمر تنفيذ هذه الشيفرة طالما كانت نتيجة التعبير المستعمل معها تساوي `true` أي طالما كان شرط ما محققًا.

يمكنك أن تتخيل أنَّ حلقة `while` هي عبارة شريطة تكرارية؛ فبعد انتهاء تنفيذ التعليمة الشرطية `if`، يُستكمل تنفيذ بقية البرنامج، لكن مع حلقة `while` فسيعود تنفيذ البرنامج إلى بداية الحلقة بعد انتهاء تنفيذها إلى أن يصبح الشرط مساويًا للقيمة `false` أي لم يعد الشرط محققًا.

وعلى النقيض من حلقات `for` التي تُنفَّذ عددًا معيَّنًا من المرات، فسيستمر تنفيذ حلقات `while` اعتمادًا على شرطٍ معيَّن، لذا لن تحتاج إلى عدد مرات تنفيذ الحلقة قبل إنشائها.

الشكل العام لحلقات `while` في لغة بايثون كالآتي:

```
while [a condition is True]:
    [do something]
```

سيستمر تنفيذ التعليمات البرمجية الموجودة داخل الحلقة إلى أن يُقَيِّم الشرط الذي يلي

`while` إلى القيمة `false`.

لننشئ برنامجًا صغيرًا فيه حلقة `while`، ففي هذه البرنامج سنطلب من المستخدم إدخال

كلمة مرور. وهناك خياران أمام حلقة التكرار:

- إما أن تكون كلمة المرور صحيحة، فعندها سينتهي تنفيذ حلقة `while`.

- أو أن تكون كلمة المرور غير صحيحة، فعندها سيستمر تنفيذ حلقة التكرار.

لننشئ ملفًا باسم `password.py` في محرّرنا النصي المفضّل، ولنبدأ بتهيئة

المتغير `password` بإسناد سلسلة نصية فارغة إليه:

```
password = ''
```

نستخدم المتغير السابق للحصول على مدخلات المستخدم داخل حلقة التكرار `while`.

علينا بعد ذلك إنشاء حلقة `while` مع تحديد ما هو الشرط الذي يجب تحقيقه:

```
password = ''
```

```
while password != 'password':
```

أتبعنا -في المثال السابق- الكلمة المحجوزة `while` بالمتغير `password`، ثمّ سنتحقق إذا

كانت قيمة المتغير `password` تساوي السلسلة النصية `'password'` (لا تنس أن قيمة المتغير

سنحصل عليها من مدخلات المستخدم)، يمكنك أن تختار أي سلسلة نصية تشاء لموازنة

مدخلات المستخدم بها. هذا يعني أنّه لو أدخل المستخدم السلسلة النصية `password` فستتوقف

حلقة التكرار وسيُكَمَّل تنفيذ البرنامج وستُنقَذ أيّة شيفرات خارج الحلقة، لكن إذا أدخل

المستخدم أيّة سلسلة نصية لا تساوي `password` فسيُكَمَّل تنفيذ الحلقة. علينا بعد ذلك إضافة

الشيفرة المسؤولة عمّا يحدث داخل حلقة `while`:

```
password = ''

while password != 'password':
    print('What is the password?')
    password = input()
```

نُفذ البرنامج عبارة `print` داخل حلقة `while` والتي تسأل المستخدم عن كلمة مروره، ثم أسندنا قيمة مدخلات المستخدم (التي حصلنا عليها عبر الدالة `input()`) إلى المتغير `password`. سيتحقق البرنامج إذا كانت قيمة المتغير `password` تساوي السلسلة النصية `'password'`، وإذا تحقق ذلك فسينتهي تنفيذ حلقة `while`. لنضف سطرًا آخر إلى البرنامج لنعرف ماذا يحدث إن أصبحت قيمة الشرط مساويةً إلى `false`:

```
password = ''

while password != 'password':
    print('What is the password?')
    password = input()

print('Yes, the password is ' + password + '. You may enter.')
```

لاحظ أنَّ آخر عبارة `print()` موجودة خارج حلقة `while`، لذا عندما يُدخِل المستخدم الكلمة `password` عند سؤاله عن كلمة مروره، فسُطْبِعَ آخر جملة والتي تقع خارج حلقة التكرار. لكن ماذا يحدث لو لم يدخل المستخدم الكلمة `password` قط؟ إذ لن يستمر تنفيذ البرنامج ولن يروا آخر عبارة `print()` وسيستمر تنفيذ حلقة التكرار إلى ما لا نهاية! يستمر تنفيذ حلقة التكرار إلى ما لا نهاية إذا بقي تنفيذ البرنامج داخل حلقة تكرار دون الخروج منها. وإذا أردت الخروج من حلقة تكرار نهائية، فاضغط `Ctrl+C` في سطر الأوامر. احفظ البرنامج ثم شغله:

```
python password.py
```

سيُطلب منك إدخال كلمة المرور، ويمكنك تجربة ما تشاء من الكلمات. هذا مثال عن

ناتج البرنامج:

```
What is the password?
hello
What is the password?
sammy
What is the password?
PASSWORD
What is the password?
password
Yes, the password is password. You may enter.
```

أبقى في ذهنك أنَّ السلاسل النصية حساسة لحالة الأحرف إلا إذا استعملت دالةً من دوال النصوص لتحويل السلسلة النصية إلى حالة الأحرف الصغيرة (على سبيل المثال) قبل التحقق منها.

## ١. تطبيق عملي

بعد أن تعلمنا المبدأ الأساسي لحلقة تكرار `while`، فلنُنشئ لعبة تعمل على سطر الأوامر لتخمين الأرقام والتي تستعمل الحلقة `while`. نريد من الحاسوب أن يُنشئ أرقامًا عشوائيةً لكي يحاول المستخدمون تخمينها، لذا علينا استيراد الوحدة `random` عبر استخدام التعليمة `import` (سنطرق لاحقًا إلى كيفية استيراد الوحدات في فصل [الوحدات](#))، وإذا لم تكن هذه الحزمة مألوفةً لك فيمكنك قراءة المزيد من المعلومات عن توليد الأرقام العشوائية في توثيق بايثون. لنُنشئ بدايةً ملفًا باسم `guess.py` في محررك النصي المفضَّل:

```
import random
```

علينا الآن إسناد عدد صحيح عشوائي إلى المتغير `number`، ولنجعل مجاله من 1 إلى 25 (بما فيها تلك الأرقام) كيلا نجعل اللعبة صعبة جدًا.

```
import random
```

```
number = random.randint(1, 25)
```

يمكننا الآن إنشاء حلقة `while`، وذلك بتهيئة متغير ثم كتابة الحلقة:

```
import random
```

```
number = random.randint(1, 25)
```

```
number_of_guesses = 0
```

```
while number_of_guesses < 5:
    print('Guess a number between 1 and 25:')
```

```
    guess = input()
    guess = int(guess)
```

```
    number_of_guesses = number_of_guesses + 1
```

```
    if guess == number:
        break
```

هنا متغيرًا اسمه `number_of_guesses` قيمته 0، وسوف نزيد قيمته عند كل تكرار

للحلقة لكي لا تصبح حلقتنا لا نهائية ثم سنضيف حلقة `while` التي تشترط ألا تزيد قيمة المتغير `number_of_guesses` عن خمس تكرارات.

وبعد المحاولة الخامسة سيُعاد المستخدم إلى سطر الأوامر، وإذا حاول المستخدم إدخال

أي شيء غير رقمي فسيحصل على رسالة خطأ.

أضفنا داخل حلقة `while` عبارة `print()` لطلب إدخال رقم من المستخدم، ثم سنأخذ

مدخلات المستخدم عبر الدالة `input()` ونُسَيِّدها إلى المتغير `guess`، ثم سنحوّل المتغير `guess`

من سلسلة نصية إلى عدد صحيح. وقبل انتهاء حلقة التكرار، فعلينا زيادة قيمة المتغير

`number_of_guesses` بمقدار 1، لكيلا تُنفَّذ حلقة التكرار أكثر من 5 مرات.

وفي النهاية، كتبنا التعليمة `if` شرطية لنرى إذا كان المتغير `guess` الذي أدخله المستخدم

مساوٍ للرقم الموجود في المتغير `number` الذي ولّده الحاسوب، وإذا تحقق الشرط فسنستخدم

التعليمة `break` للخروج من الحلقة. أصبح البرنامج جاهزًا للاستخدام، ويمكننا تشغيله عبر تنفيذ

الأمر التالي:

```
python guess.py
```

صحيحٌ أنَّ البرنامج يعمل عملاً سليماً، لكن المستخدم لن يعلم إذا كان تخمينه صحيحاً

ويمكنه أن يخمّن الرقم خمس مرات دون أن يعلم إذا كانت إحدى محاولاته صحيحة. هذا مثال

عن مخرجات البرنامج:

```
Guess a number between 1 and 25:
11
Guess a number between 1 and 25:
19
Guess a number between 1 and 25:
22
Guess a number between 1 and 25:
3
```

Guess a number between 1 and 25:

8

لنصف بعض التعليمات الشرطية خارج حلقة التكرار لكي يحصل المستخدم على معلومات

فيما إذا استطاعوا تخمين الرقم أم لا، وسنضيف هذه العبارات في نهاية الملف:

```
import random

number = random.randint(1, 25)

number_of_guesses = 0

while number_of_guesses < 5:
    print('Guess a number between 1 and 25:')
    guess = input()
    guess = int(guess)

    number_of_guesses = number_of_guesses + 1

    if guess == number:
        break

if guess == number:
    print('You guessed the number in ' + str(number_of_guesses)
    + ' tries!')

else:
    print('You did not guess the number. The number was ' +
    str(number))
```

سيُخبر البرنامج في هذه المرحلة المستخدم إذا استطاعوا تخمين الرقم، لكن ذلك لن

يحدث إلا بعد انتهاء حلقة التكرار وبعد انتهاء عدد مرات التخمين المسموحة. ولمساعد

المستخدم قليلاً، فلنضف بعض التعليمات الشرطية داخل حلقة `while` وستخبر تلك التعليمات المستخدم إذا كان تخمينه أعلى من الرقم أو أصغر منه، لكي يستطيعوا تخمين الرقم بنجاح، وسنضيف تلك التعليمات الشرطية قبل السطر الذي يحتوي على `if guess == number`

```
import random

number = random.randint(1, 25)
number_of_guesses = 0

while number_of_guesses < 5:
    print('Guess a number between 1 and 25:')
    guess = input()
    guess = int(guess)

    number_of_guesses = number_of_guesses + 1

    if guess < number:
        print('Your guess is too low')

    if guess > number:
        print('Your guess is too high')

    if guess == number:
        break

if guess == number:
    print('You guessed the number in ' + str(number_of_guesses)
    + ' tries!')

else:
    print('You did not guess the number. The number was ' +
    str(number))
```

وعندما نُشغّل البرنامج مرةً أخرى بتنفيذ `python guess.py`، فيمكننا ملاحظة أنَّ المستخدم سيحصل على بعض المساعدة، فلو كان الرقم المولّد عشوائيًا هو 12 وكان تخمين المستخدم 18، فسيُخبره البرنامج أنَّ الرقم الذي خمنه أكبر من الرقم العشوائي، وذلك لكي يستطيع تعديل تخمينه وفقًا لذلك. هنالك الكثير من التحسينات التي يمكن إجراؤها على الشيفرة السابقة، مثل تضمين آلية لمعالجة الأخطاء التي تحدث عندما لا يُدخّل المستخدم عددًا صحيحًا، لكن كان غرضنا هو رؤية كيفية استخدام حلقة `while` في برنامج قصير ومفيد يعمل من سطر الأوامر.

## 2. حلقة التكرار `for`

حلقة `for` تؤدي إلى تكرار تنفيذ جزء من الشيفرات بناءً على عدّد أو على متغيّر، وهذا يعني أنَّ حلقات `for` تستعمل عندما يكون عدد مرات تنفيذ حلقة التكرار معلومًا قبل الدخول في الحلقة، وذلك على النقيض من حلقات `while` المبنية على شرط. تُبنى حلقات `for` في بايثون كما يلي:

```
for [iterating variable] in [sequence]:
    [do something]
```

سُتنفذ الشيفرات الموجودة داخل حلقة التكرار عدّة مرات إلى أن تنتهي الحلقة. لننظر إلى

كيفية مرور الحلقة `for` على مجالٍ من القيم:

```
for i in range(0,5):
    print(i)
```

سيُخرج البرنامج السابق عند تشغيله الناتج الآتي:

0  
1  
2  
3  
4

ضبطنا المتغير `i` في حلقة `for` ليحتوي على القيمة التي سَنُقَدِّمُ عليها حلقة التكرار، وكان مجال القيم التي سَنُسَدِّدُ إلى هذا المتغير من 0 إلى 5. ثم طُبِعَ قيمة المتغير في كل دوران لحلقة التكرار، لكن أبقِ في ذهنك أننا نميل إلى بدء العد من الرقم 0 في البرمجة، وعلى الرغم من عرض خمسة أرقام، لكنها تبدأ بالرقم 0 وتنتهي بالرقم 4. من الشائع أن ترى استخدامًا لحلقة `for` عندما تحتاج إلى تكرار كتلة معيّنة من الشيفرات لعددٍ من المرات.

### ١. استخدام حلقة التكرار `for` مع الدالة `range()`

إحدى أنواع السلاسل غير القابلة للتعديل في بايثون هي تلك الناتجة من الدالة `range()`، وتستخدم الدالة `range()` في حلقات التكرار للتحكم بعدد مرات تكرار الحلقة. عند التعامل مع الدالة `range()` عليك أن تمرر معاملاً رقميًا أو معاملين أو ثلاثة معاملات:

- `start`: يشير إلى القيم العددية الصحيحة التي ستبدأ بها السلسلة، وإذا لم تُمرَّر قيمة لهذا المعامل فستبدأ السلسلة من 0.
- `stop`: هذا المعامل مطلوب دومًا وهو القيمة العددية الصحيحة التي تمثل نهاية السلسلة العددية لكن دون تضمينها.
- `step`: هي مقدار الخطوة، أي عدد الأرقام التي يجب زيادتها (أو إنقاصها إن كنّا نتعامل مع أرقام سالبة) في الدورة القادمة، وقيمة المعامل `step` تساوي 1 إن لم تُحدَّد له قيمة.

لننظر إلى بعض الأمثلة التي تُمرَّر فيها مختلف المعاملات إلى الدالة `range()`. لنبدأ بتمرير

المعامل `stop` فقط، أي أنَّ السلسلة الآتية من الشكل `range(stop)`:

```
for i in range(6):
    print(i)
```

كانت قيمة المعامل `stop` في المثال السابق مساويةً للرقم 6، لذا ستمر حلقة التكرار من

بداية المجال 0 إلى نهايته 6 (باستثناء الرقم 6 كما ذكرنا أعلاه):

```
0
1
2
3
4
5
```

المثال الآتي من الشكل `range(start , stop)` الذي تُمرَّر قيم بدء السلسلة ونهايتها:

```
for i in range(20,25):
    print(i)
```

المجال -في المثال السابق- يتراوح بين 20 (بما فيها الرقم 20) إلى 25 (باستثناء الرقم

25)، لذا سيبدو الناتج كما يلي:

```
20
21
22
23
24
```

الوسيط `step` الخاص بالدالة `range()` شبيه بمعامل الخطوة الذي نستعمله عند تقسيم

السلاسل النصية لأنه يستعمل لتجاوز بعض القيم ضمن السلسلة. يأتي المعامل `step` في آخر

قائمة المعاملات التي تقبلها الدالة `range()` وذلك بالشكل `range(start, stop, step)`.  
لنستعمل المعامل `step` مع قيمة موجبة:

```
for i in range(0,15,3):
    print(i)
```

سيؤدي المثال السابق إلى إنشاء سلسلة من الأرقام التي تبدأ من 0 وتنتهي عند 15 لكن قيمة المعامل `step` هي 3، لذا سيتم تخطي رقمين في كل دورة، أي سيكون الناتج كالاتي:

```
0
3
6
9
12
```

يمكننا أيضًا استخدام قيمة سالبة للمعامل `step` للدوران إلى الخلف، لكن علينا تعديل قيم `start` و `stop` بما يتوافق مع ذلك:

```
for i in range(100,0,-10):
    print(i)
```

قيمة المعامل `start` في المثال السابق هي 100، وكانت قيمة المعامل `stop` هي 0، والخطوة هي -10، لذا ستبدأ السلسلة من الرقم 100 وستنتهي عند الرقم 0، وسيكون التناقص بمقدار 10 في كل دورة، ويمكننا ملاحظة ذلك في الناتج الآتي:

```
100
90
80
70
60
```

50  
40  
30  
20  
10

عندما نبرمج باستخدام لغة بايثون، فسنجد أننا نستفيد كثيرًا من السلاسل الرقمية التي

تنتجها الدالة `range()`.

### ب. استخدام حلقة `for` مع أنواع البيانات المتسلسلة

يمكن الاستفادة من القوائم (من النوع `list`) وغيرها من أنواع البيانات المتسلسلة

واستعمالها بعدّها معاملات لحلقات `for`، بدلاً من الدوران باستخدام الدالة `range()` فيمكننا

تعريف قائمة ثم الدوران على عناصرها. سنُعيد في المثال الآتي قائمةً إلى متغيّر، ثم سنستخدم

حلقة `for` للدوران على عناصر القائمة:

```
sharks = ['hammerhead', 'great white', 'dogfish', 'frilled',
          'bullhead', 'requiem']

for shark in sharks:
    print(shark)
```

في هذه الحالة، طبعنا كل عنصر موجود في القائمة؛ وصحيحٌ أننا استعملنا الكلمة `shark`

اسمًا للمتغيّر، لكن يمكنك استعمال أي اسم صحيح آخر ترغب به، وستحصل على نفس النتيجة.

ناتج تنفيذ المثال السابق هو:

```
hammerhead
great white
```

```
dogfish
frilled
bullhead
requiem
```

الناتج السابق يُظهر دوران الحلقة for على جميع عناصر القائمة مع طباعة كل عنصر في سطرٍ منفصل. يشجع استخدام القوائم والأنواع الأخرى من البيانات المتسلسلة مثل السلاسل النصية وبني tuple (الصفوف) مع حلقات التكرار لسهولة الدوران على عناصرها. يمكنك دمج هذه الأنواع من البيانات مع الدالة range() لإضافة عناصر إلى قائمة، مثل:

```
sharks = ['hammerhead', 'great white', 'dogfish', 'frilled',
          'bullhead', 'requiem']

for item in range(len(sharks)):
    sharks.append('shark')

print(sharks)
```

الناتج:

```
['hammerhead', 'great white', 'dogfish', 'frilled', 'bullhead',
 'requiem', 'shark', 'shark', 'shark', 'shark', 'shark',
 'shark']
```

أضفنا هنا السلسلة النصية 'shark' خمس مرات (وهو نفس طول القائمة sharks الأصلي) إلى القائمة sharks.

يمكننا استخدام حلقة for لبناء قائمة جديدة:

```
integers = []
```

```
for i in range(10):
    integers.append(i)
print(integers)
```

ههنا في المثال السابق قائمةً فارغةً باسم `integers` لكن حلقة التكرار `for` ملأت القائمة

لتصبح كما يلي:

```
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

وبشكلٍ شبيهٍ بما سبق، يمكننا الدوران على السلاسل النصية:

```
sammy = 'Sammy'

for letter in sammy:
    print(letter)
```

النتاج:

```
S
a
m
m
y
```

يمكن الدوران على بنى `tuple` كما هو الحال في القوائم والسلاسل النصية. عند المرور على

عناصر نوع البيانات `dictionary`، فمن المهم أن تبقي بذهنك البنية الخاصة به «مفتاح:قيمة»

(`key:value`) لكي تضمن أنك تستدعي العنصر الصحيح من المتغير.

إليك مثالٌ بسيطٌ نعرض فيه المفتاح (`key`) والقيمة (`value`):

```
sammy_shark = {'name': 'Sammy', 'animal': 'shark', 'color':
'blue', 'location': 'ocean'}
```

```
for key in sammy_shark:
    print(key + ': ' + sammy_shark[key])
```

الناتج:

```
name: Sammy
animal: shark
location: ocean
color: blue
```

عند استخدام متغيرات من النوع dictionary (قاموس) مع حلقات for فيكون المتغير المرتبط بحلقة التكرار متعلقًا بمفتاح القيم، وعلينا استخدام الصياغة `dictionary_variable[iterating_variable]` للوصول إلى القيمة الموافقة للمفتاح. ففي المثال السابق كان المتغير المرتبط بحلقة التكرار باسم `key` وهو يُمثل المفاتيح، واستعملنا `sammy_shark[key]` للوصول إلى القيمة المرتبطة بذاك المفتاح. خلاصة ما سبق، تُستعمل حلقات التكرار عادةً للدوران على عناصر البيانات المتسلسلة وتعديلها.

### ج. حلقات for المتشعبة

يمكن تشعب حلقات التكرار في بايثون، كما هو الحال في بقية لغات البرمجة. حلقة التكرار المتشعبة هي الحلقة الموجودة ضمن حلقة تكرار أخرى، وهي شبيهة بعبارات `if` المتشعبة. تُبنى حلقات التكرار المتشعبة كما يلي:

```
# الحلقة الخارجية
for [first iterating variable] in [outer loop]:
    [do something] # اختياري
```

## # الحلقة الداخلية الفرعية

```
for [second iterating variable] in [nested loop]:
    [do something]
```

يبدأ البرنامج بتنفيذ حلقة التكرار الخارجية، ويُنفَّذ أوَّل دوران فيها، وأوَّل دوران سيؤدي إلى الدخول إلى حلقة التكرار الداخلية، مما يؤدي إلى تنفيذها إلى أن تنتهي تمامًا. ثم سيعود تنفيذ البرنامج إلى بداية حلقة التكرار الخارجية، ويبدأ بتنفيذ الدوران الثاني، ثم سيصل التنفيذ إلى حلقة التكرار الداخلية، وستُنَفَّذ حلقة التكرار الداخلية بالكامل، ثم سيعود التنفيذ إلى بداية حلقة التكرار الخارجية، وهلم جرا إلى أن ينتهي تنفيذ حلقة التكرار الخارجية أو إيقاف حلقة التكرار عبر استخدام التعليمة break أو غيرها. لُنَشِئْ مثالًا يستعمل حلقة for متشعبة لكي نفهم كيف تعمل بدقة، إذ ستمر حلقة التكرار الخارجية في المثال الآتي على قائمة من الأرقام اسمها num\_list، أما حلقة التكرار الداخلية فستمر على قائمة من السلاسل النصية اسمها alpha\_list:

```
num_list = [1, 2, 3]
alpha_list = ['a', 'b', 'c']

for number in num_list:
    print(number)
    for letter in alpha_list:
        print(letter)
```

سيظهر الناتج الآتي عند تشغيل البرنامج:

```
1
a
b
c
```

```
2
a
b
c
3
a
b
c
```

يُظهر الناتج السابق أنَّ البرنامج أكمل أوَّل دوران على عناصر حلقة التكرار الخارجية بطباعة الرقم 1، ومن ثم بدأ تنفيذ حلقة التكرار الداخلية مما يطبع الأحرف a و b و c على التوالي. وبعد انتهاء تنفيذ حلقة التكرار الداخلية، عاد البرنامج إلى بداية حلقة التكرار الخارجية طابعًا الرقم 2، ثم بدأ تنفيذ حلقة التكرار الداخلية (مما يؤدي إلى إظهار a و b و c مجددًا). وهكذا دواليك.

يمكن الاستفادة من حلقات for المتشعبة عند المرور على عناصر قوائم تتألف من قوائم. فلو استعملنا حلقة تكرار وحيدة لعرض عناصر قائمة تتألف من عناصر تحتوي على قوائم، فسُتعرض قيم القوائم الداخلية:

```
list_of_lists = [['hammerhead', 'great white', 'dogfish'], [0,
1, 2], [9.9, 8.8, 7.7]]

for list in list_of_lists:
    print(list)
```

الناتج:

```
['hammerhead', 'great white', 'dogfish']
[0, 1, 2]
```

```
[9.9, 8.8, 7.7]
```

وفي حال أردنا الوصول إلى العناصر الموجودة في القوائم الداخلية، فيمكننا استعمال حلقة

for متشعبة:

```
list_of_lists = [['hammerhead', 'great white', 'dogfish'], [0,
1, 2], [9.9, 8.8, 7.7]]

for list in list_of_lists:
    for item in list:
        print(item)
```

النتائج:

```
hammerhead
great white
dogfish
0
1
2
9.9
8.8
7.7
```

نستطيع الاستفادة من حلقات for المتشعبة عندما نريد الدوران على عناصر محتوى

في قوائم.

### 3. التحكم بحلقات التكرار

وجدنا أنَّ استخدام حلقات for أو while تسمح بأتمتة وتكرار المهام بطريقة فعّالة. لكن

في بعض الأحيان، قد يتدخل عامل خارجي في طريقة تشغيل برنامجك، وعندما يحدث ذلك،

ربما تريد من برنامجك الخروج تمامًا من حلقة التكرار، أو تجاوز جزء من الحلقة قبل إكمال تنفيذها، أو تجاهل هذا العامل الخارجي تمامًا. لذا يمكنك فعل ما سبق باستخدام التعليمات `break` و `continue` و `pass`.

### ١. التعليمة `break`

توفّر لك التعليمة `break` القدرة على الخروج من حلقة التكرار عند حدوث عامل خارجي. فعليك وضع التعليمة `break` في الشيفرة التي ستُنقذ في كل تكرار للحلقة، وتوضع عادةً ضمن تعليمة شرطية مثل `if`. ألق نظرةً إلى أحد الأمثلة الذي يستعمل التعليمة `break` داخل حلقة `for`:

```
number = 0

for number in range(10):
    number = number + 1 (*)

    if number == 5:
        break # توقف هنا

    print('Number is ' + str(number))

print('Out of loop')
```

هذا برنامجٌ صغيرٌ، هيأنا في بدايته المتغير `number` بجعله يساوي الصفر، ثم بنينا حلقة تكرار `for` التي تعمل لطالما كانت قيمة المتغير `number` أصغر من 10. ثم قمنا بزيادة قيمة المتغير `number` داخل حلقة `for` بمقدار 1 في كل تكرار، وذلك في السطر (\*) ثم كان هنالك الشرط `if` الذي يختبر إن كان المتغير `number` مساوٍ للرقم 5، وعند حدوث ذلك فسُتُنقذ التعليمة

`break` للخروج من الحلقة، وتوجد داخل حلقة التكرار الدالة `print()` التي تُنفَّذ في كل تكرار إلى أن نخرج من الحلقة عبر التعليمة `break`، إذ هي موجودة بعد التعليمة `break`. لكي نتأكد أننا خرجنا من الحلقة، وضعنا عبارة `print()` أخيرة موجودة خارجها. سنرى الناتج الآتي عند تنفيذ البرنامج:

```
Number is 1
Number is 2
Number is 3
Number is 4
Out of loop
```

يُظهر الناتج السابق أنه بمجرد أن أصبح العدد الصحيح `number` مساوياً للرقم 5، فسيُنْتهى تنفيذ حلقة التكرار عبر `break`.

### ب. التعليمة `continue`

تسمح لنا التعليمة `continue` بتخطي جزء من حلقة التكرار عند حدوث عامل خارجي، وعدم إكمال بقية الحلقة إلى نهايتها. بعبارةٍ أخرى، سينتقل تنفيذ البرنامج إلى أول حلقة التكرار عند تنفيذ التعليمة `continue`. يجب وضع التعليمة `continue` في الشيفرة التي سَتُنْفَّذ في كل تكرار للحلقة، ويوضع عادةً ضمن الشرط `if`.

سنستخدم نفس البرنامج الذي استعملناها لشرح التعليمة `break` أعلاه، لكننا سنستخدم

التعليمة `continue` بدلاً من `break`:

```
number = 0

for number in range(10):
    number = number + 1
```

```

if number == 5:
    continue # continue here

print('Number is ' + str(number))

print('Out of loop')

```

الفرق بين استخدام التعليمة `continue` بدلاً من `break` هو إكمال تنفيذ الشيفرة بغض النظر عن التوقف الذي حدث عندما كانت قيمة المتغير `number` مساويةً إلى الرقم 5. لننظر إلى الناتج:

```

Number is 1
Number is 2
Number is 3
Number is 4
Number is 6
Number is 7
Number is 8
Number is 9
Number is 10
Out of loop

```

نلاحظ أنَّ السطر الذي يجب أن يحتوي على `Number is 5` ليس موجوداً في المخرجات، لكن سيُكَمَّل تنفيذ حلقة التكرار بعد هذه المرحلة مما يطبع الأرقام من 6 إلى 10 قبل إنهاء تنفيذ الحلقة.

يمكنك استخدام التعليمة `continue` لتفادي استخدام تعليمات شرطية معقَّدة ومتشعِّبة، أو لتحسين أداء البرنامج عن طريق تجاهل الحالات التي سترفض نتائجها.

**ج. التعليمة pass**

تسمح لنا التعليمة pass بالتعامل مع أحد الشروط دون إيقاف عمل حلقة التكرار بأي شكل، أي سنُنفِّذ جميع التعليمات البرمجية الموجودة في حلقة التكرار ما لم تستعمل تعليمات تحكم مثل break أو continue فيها. وكما هو الحال مع التعليمات السابقة، يجب وضع التعليمة pass في الشيفرة التي سنُنفِّذ في كل تكرار للحلقة، ويوضع عادةً ضمن الشرط if. سنستخدم نفس البرنامج الذي استعملناها لشرح التعليمة break أو continue أعلاه، لكننا سنستخدم التعليمة pass هذه المرة:

```
number = 0

for number in range(10):
    number = number + 1

    if number == 5:
        pass # تخطى وأكمل

    print('Number is ' + str(number))

print('Out of loop')
```

تخبر التعليمة pass التي تقع بعد الشرط if البرنامج أنَّ عليه إكمال تنفيذ الحلقة وتجاهل مساواة المتغير number للرقم 5. لنشغِّل البرنامج ولننظر إلى الناتج:

```
Number is 1
Number is 2
Number is 3
Number is 4
```

```
Number is 5
Number is 6
Number is 7
Number is 8
Number is 9
Number is 10
Out of loop
```

لاحظنا عند استخدامنا للتعليلة `pass` في هذا البرنامج أنَّ البرنامج يعمل كما لو أننا لم نضع تعليلة شرطية داخل حلقة التكرار؛ إذ تخبر التعليلة `pass` البرنامج أن يكمل التنفيذ كما لو أنَّ الشرط لم يتحقق. يمكن أن تستفيد من التعليلة `pass` عندما تكتب برنامجك لأول مرة أثناء تفكيرك بحلّ مشكلة ما عبر خوارزمية، لكن قبل أن تضع التفاصيل التقنية له.

## 4. خلاصة الفصل

شرحنا في هذا الفصل كيف تعمل حلقتي التكرار `while`، و `for` في بايثون وكيفية إنشائها، إذ تستمر الأولى بتنفيذ مجموعة من الأسطر البرمجية طالما كان الشرط مساويًا للقيمة `true` بينما تستمر الثانية بتنفيذ مجموعة من الشيفرات لعددٍ مُحدّدٍ من المرات. انتقلنا أخيرًا إلى التعليمات `break` و `continue` و `pass` التي تسمح بالتحكم أكثر بحلقات `for` و `while` والتعامل مع الأحداث المفاجئة التي تحدث أثناء تنفيذ مهمة مُتكرّرة.